

✓ Dask DataFrame

Этот ноутбук повторяет структуру и логику практического задания по PySpark, но использует библиотеку **Dask** для параллельных вычислений в Python. Dask предоставляет интерфейсы, похожие на Pandas (DataFrame), NumPy (Array) и стандартные Python-коллекции (Bag), но позволяет работать с данными, которые не помещаются в оперативную память, за счёт ленивых вычислений и разбиения на блоки (partition).

Ключевое отличие от PySpark: Dask теснее интегрирован с экосистемой Python (Pandas, NumPy, Scikit-learn) и часто использует похожий синтаксис, но работает поверх них, добавляя возможности параллелизма и работы с большими данными. Для выполнения вычислений необходимо явно вызывать методы `.compute()` или `.persist()`.

✓ 1. Установка и импорт библиотек

Dask можно установить через `pip` или `conda`. Для работы с DataFrame нам понадобятся также `pandas`, `numpy` и библиотеки для визуализации.

Рекомендуется установить полный набор зависимостей:

```
!pip install "dask[complete]" # Установит Dask, Pandas, NumPy и другие зависимости
!pip install seaborn matplotlib # Библиотеки для визуализации
```

```
Requirement already satisfied: packaging<20.0,>=20.0 in /usr/local/lib/python3.12/dist-packages (from dask[complete]) (20.0)
Requirement already satisfied: partd<1.4.0,>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from dask[complete]) (1.4.2)
Requirement already satisfied: pyyaml<5.3.1,>=5.3.1 in /usr/local/lib/python3.12/dist-packages (from dask[complete]) (6.0.3)
Requirement already satisfied: toolz<0.12.0,>=0.12.0 in /usr/local/lib/python3.12/dist-packages (from dask[complete]) (0.12.1)
Requirement already satisfied: pyarrow<14.0.1,>=14.0.1 in /usr/local/lib/python3.12/dist-packages (from dask[complete]) (18.1.0)
Collecting lz4<4.3.2,>=4.3.2 (from dask[complete])
  Downloading lz4-4.4.5-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (3.8 kB)
Requirement already satisfied: locket in /usr/local/lib/python3.12/dist-packages (from partd<1.4.0,>=1.4.0->dask[complete]) (1.0.0)
Requirement already satisfied: numpy<1.24,>=1.24 in /usr/local/lib/python3.12/dist-packages (from dask[complete]) (2.0.2)
Requirement already satisfied: pandas<2.0,>=2.0 in /usr/local/lib/python3.12/dist-packages (from dask[complete]) (2.2.2)
Collecting distributed<2025.12.1,>=2025.12.0 (from dask[complete])
  Downloading distributed-2025.12.0-py3-none-any.whl.metadata (3.4 kB)
Requirement already satisfied: bokeh<3.1.0,>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from dask[complete]) (3.7.3)
Requirement already satisfied: Jinja2<2.10.3,>=2.10.3 in /usr/local/lib/python3.12/dist-packages (from dask[complete]) (3.1.6)
Requirement already satisfied: contourpy<1.2,>=1.2 in /usr/local/lib/python3.12/dist-packages (from bokeh<3.1.0,>=3.1.0->dask[complete]) (1.0.7)
Requirement already satisfied: narwhals<1.13,>=1.13 in /usr/local/lib/python3.12/dist-packages (from bokeh<3.1.0,>=3.1.0->dask[complete]) (1.0.7)
Requirement already satisfied: pillow<7.1.0,>=7.1.0 in /usr/local/lib/python3.12/dist-packages (from bokeh<3.1.0,>=3.1.0->dask[complete]) (9.4.0)
Requirement already satisfied: tornado<6.2,>=6.2 in /usr/local/lib/python3.12/dist-packages (from bokeh<3.1.0,>=3.1.0->dask[complete]) (6.2)
Requirement already satisfied: xyzservices<2021.09.1,>=2021.09.1 in /usr/local/lib/python3.12/dist-packages (from bokeh<3.1.0,>=3.1.0->dask[complete]) (2021.9.1)
Requirement already satisfied: msgpack<1.0.2,>=1.0.2 in /usr/local/lib/python3.12/dist-packages (from distributed<2025.12.1,>=2025.12.0->dask[complete]) (1.0.7)
Requirement already satisfied: psutil<5.8.0,>=5.8.0 in /usr/local/lib/python3.12/dist-packages (from distributed<2025.12.1,>=2025.12.0->dask[complete]) (5.8.0)
Collecting sortedcontainers<2.0.5,>=2.0.5 (from distributed<2025.12.1,>=2025.12.0->dask[complete])
  Downloading sortedcontainers-2.4.0-py2.py3-none-any.whl.metadata (10 kB)
Collecting tblib<3.2.0,!=3.2.1,>=1.6.0 (from distributed<2025.12.1,>=2025.12.0->dask[complete])
  Downloading tblib-3.2.2-py3-none-any.whl.metadata (27 kB)
Requirement already satisfied: urllib3<1.26.5,>=1.26.5 in /usr/local/lib/python3.12/dist-packages (from distributed<2025.12.1,>=2025.12.0->dask[complete]) (1.26.5)
Collecting zict<3.0.0,>=3.0.0 (from distributed<2025.12.1,>=2025.12.0->dask[complete])
  Downloading zict-3.0.0-py2.py3-none-any.whl.metadata (899 bytes)
Requirement already satisfied: MarkupSafe<2.0,>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2<2.10.3,>=2.10.3->dask[complete]) (2.0.1)
Requirement already satisfied: python-dateutil<2.8.2,>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas<2.0,>=2.0->dask[complete]) (2.8.2)
Requirement already satisfied: pytz<2020.1,>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas<2.0,>=2.0->dask[complete]) (2020.1)
Requirement already satisfied: tzdata<2022.7,>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas<2.0,>=2.0->dask[complete]) (2022.7)
Requirement already satisfied: six<1.5,>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil<2.8.2,>=2.8.2->pandas<2.0,>=2.0->dask[complete]) (1.16.0)
Downloaded lz4-4.4.5-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (1.4 MB)
1.4/1.4 MB 56.4 MB/s eta 0:00:00
Downloaded distributed-2025.12.0-py3-none-any.whl (1.0 MB)
1.0/1.0 MB 56.9 MB/s eta 0:00:00
Downloaded sortedcontainers-2.4.0-py2.py3-none-any.whl (29 kB)
Downloaded tblib-3.2.2-py3-none-any.whl (12 kB)
Downloaded zict-3.0.0-py2.py3-none-any.whl (43 kB)
43.3/43.3 kB 2.9 MB/s eta 0:00:00
Installing collected packages: sortedcontainers, zict, tblib, lz4, distributed
Successfully installed distributed-2025.12.0 lz4-4.4.5 sortedcontainers-2.4.0 tblib-3.2.2 zict-3.0.0
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-packages (0.13.2)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: numpy<1.24.0,>=1.20 in /usr/local/lib/python3.12/dist-packages (from seaborn) (2.0.2)
Requirement already satisfied: pandas<1.2,>=1.2 in /usr/local/lib/python3.12/dist-packages (from seaborn) (2.2.2)
Requirement already satisfied: contourpy<1.0.1,>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler<0.10,>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools<4.22.0,>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.61.1)
Requirement already satisfied: kiwisolver<1.3.1,>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.4.9)
Requirement already satisfied: packaging<20.0,>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (25.0)
Requirement already satisfied: pillow<8,>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
```

```
# Импорт основных библиотек
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Настройка отображения графиков в ноутбуке
%matplotlib inline
sns.set_style("whitegrid")
```

```
# Импорт Dask
import dask
import dask.dataframe as dd # Основной модуль для работы с DataFrame
from dask.diagnostics import ProgressBar # Для отображения прогресса выполнения
```

Примечание: В отличие от PySpark, для Dask не требуется создавать отдельный сеанс (SparkSession). Вычисления можно запускать сразу после импорта.

2. Загрузка данных

Из списка кортежей или словарей

Сначала создадим обычный Pandas DataFrame, а затем преобразуем его в Dask DataFrame. Dask может работать с небольшими данными в памяти, разбивая их на указанное количество разделов (partitions).

```
# Создаем Pandas DataFrame из списка кортежей
data = [
    ('Bob', 'Galager', 90, 30),
    ('Fido', '-', 30, 5),
    ('Kristian', 'Bale', 90, 50)
]
columns = ['name', 'lastname', 'weight', 'age']

pandas_df = pd.DataFrame(data=data, columns=columns)

# Преобразуем в Dask DataFrame с одним разделом (npartitions=1)
dask_df_from_tuples = dd.from_pandas(pandas_df, npartitions=1)
print("Тип объекта:", type(dask_df_from_tuples))
dask_df_from_tuples.compute() # .compute() выполняет вычисления и возвращает Pandas DataFrame
```

Тип объекта: <class 'dask.dataframe.dask_expr._collection.DataFrame'>

	name	lastname	weight	age
0	Bob	Galager	90	30
1	Fido	-	30	5
2	Kristian	Bale	90	50

```
# Создаем Dask DataFrame напрямую из списка словарей (через Pandas)
data_dict = [
    {'name': 'Bob', 'lastname': 'Galager', 'weight': 90, 'age': 30},
    {'name': 'Fido', 'lastname': '-', 'weight': 30, 'age': 5},
    {'name': 'Kristian', 'lastname': 'Bale', 'weight': 90, 'age': 50}
]

dask_df_from_dict = dd.from_pandas(pd.DataFrame(data_dict), npartitions=1)
dask_df_from_dict.compute()
```

	name	lastname	weight	age
0	Bob	Galager	90	30
1	Fido	-	30	5
2	Kristian	Bale	90	50

Из файла

Dask поддерживает чтение из множества форматов (CSV, Parquet, JSON, ORC и др.) и источников (локальный диск, Amazon S3, Google Cloud Storage).

```
# Загрузка данных о пингвинах. Используем встроенный пример, так как файл 'penguins.csv' может отсутствовать.
# Если у вас есть файл 'penguins.csv', используйте: df = dd.read_csv('penguins.csv')

# Для демонстрации создадим синтетические данные через dask.datasets
# Вместо этого в реальной задаче вы бы использовали dd.read_csv('penguins.csv')

# Создаем небольшой синтетический датасет
penguins_data = {
    'species': ['Adelie', 'Adelie', 'Gentoo', 'Chinstrap', 'Adelie'] * 20,
    'island': ['Torgersen', 'Biscoe', 'Dream', 'Dream', 'Torgersen'] * 20,
    'bill_length_mm': np.random.uniform(35, 50, 100),
    'bill_depth_mm': np.random.uniform(15, 20, 100),
    'flipper_length_mm': np.random.randint(170, 210, 100),
    'body_mass_g': np.random.randint(3000, 5000, 100),
    'sex': np.random.choice(['MALE', 'FEMALE', None], 100, p=[0.45, 0.45, 0.1])
}

# Преобразуем в Pandas, затем в Dask DataFrame
penguins_pandas_df = pd.DataFrame(penguins_data)
# Добавим немного пропусков для реалистичности в числовых столбцах
mask = np.random.choice([True, False], size=penguins_pandas_df.shape, p=[0.02, 0.98])
penguins_pandas_df = penguins_pandas_df.where(~mask)

df = dd.from_pandas(penguins_pandas_df, npartitions=4) # Разобьем на 4 раздела

# Визуализируем структуру Dask DataFrame (ленивая операция, без .compute())
print("Структура Dask DataFrame (не выполненная):")
print(df)
```

```
Структура Dask DataFrame (не выполненная):
Dask DataFrame Structure:
              species  island bill_length_mm bill_depth_mm flipper_length_mm body_mass_g      sex
npartitions=4
0              string   string         float64         float64              float64    float64  string
25                ...     ...             ...             ...                ...         ...     ...
50                ...     ...             ...             ...                ...         ...     ...
75                ...     ...             ...             ...                ...         ...     ...
99                ...     ...             ...             ...                ...         ...     ...
Dask Name: frompandas, 1 expression
Expr=df
```

```
# Чтобы увидеть данные, нужно выполнить вычисление
with ProgressBar():
    df_head = df.head(5)
print("Первые 5 строк:")
print(df_head)
```

```
[#####] | 100% Completed | 106.03 ms
Первые 5 строк:
   species  island  bill_length_mm  bill_depth_mm  flipper_length_mm  \
0    <NA>  Torgersen    49.819973    16.987646         204.0
1  Adelie  Biscoe    38.599881    17.102706         204.0
2  Gentoo  Dream    39.414298    17.681383         176.0
3 Chinstrap  Dream    41.954704    17.736860         199.0
4  Adelie  Torgersen    35.120439    19.420192         178.0

   body_mass_g  sex
0    3565.0    <NA>
1    4114.0    <NA>
2    4111.0  FEMALE
3    3531.0   MALE
4    3662.0   MALE
```

✓ 3. Просмотр данных

✓ Просмотр нескольких строк

Аналогично `df.limit(3).show()` в PySpark или `df.head(3)` в Pandas.

```
# Просмотр первых N строк (аналог .limit())
with ProgressBar():
    df_head = df.head(3)
print("Первые 3 строки (аналог limit):")
print(df_head)
```

```
[#####] | 100% Completed | 106.61 ms
Первые 3 строки (аналог limit):
   species  island  bill_length_mm  bill_depth_mm  flipper_length_mm  \
0    <NA>  Torgersen    49.819973    16.987646         204.0
1  Adelie  Biscoe    38.599881    17.102706         204.0
2  Gentoo  Dream    39.414298    17.681383         176.0
```

```

    body_mass_g    sex
0      3565.0     <NA>
1      4114.0     <NA>
2      4111.0    FEMALE

```

▼ Вывод размеров датасета

В отличие от Pandas, получение количества строк в Dask — ленивая операция, требующая `.compute()`.

```

# Получаем количество строк и столбцов
num_cols = len(df.columns)
print(f"Количество столбцов: {num_cols}")

# Количество строк требует вычислений
with ProgressBar():
    num_rows = len(df) # Альтернатива: df.shape[0].compute()
print(f"Количество строк: {num_rows}")
print(f"Размер данных: ({num_rows}, {num_cols})")

Количество столбцов: 7
[#####] | 100% Completed | 106.52 ms
Количество строк: 100
Размер данных: (100, 7)

```

▼ Просмотр типов столбцов

Аналог `df.printSchema()` в PySpark или `df.info()` в Pandas.

```

# Метод .dtypes возвращает типы данных (лениво)
print("Типы данных столбцов:")
print(df.dtypes)

# Более подробная информация (аналог .info())
# В Dask нет прямого аналога .info(), поэтому можем вывести .dtypes и .columns
print("\nИмена столбцов:")
print(df.columns.tolist())

Типы данных столбцов:
species          string[pyarrow]
island           string[pyarrow]
bill_length_mm   float64
bill_depth_mm    float64
flipper_length_mm float64
body_mass_g      float64
sex             string[pyarrow]
dtype: object

Имена столбцов:
['species', 'island', 'bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', 'body_mass_g', 'sex']

```

▼ Вывод количества пропусков по столбцам

Аналог `df.isna().sum()` в Pandas.

```

# Подсчет пропусков
with ProgressBar():
    null_counts = df.isnull().sum().compute()
print("Количество пропусков по столбцам:")
print(null_counts)

[#####] | 100% Completed | 105.36 ms
Количество пропусков по столбцам:
species          2
island           5
bill_length_mm   2
bill_depth_mm    2
flipper_length_mm 3
body_mass_g      3
sex             16
dtype: int64

```

▼ Распределение целевой переменной

Построим распределение видов пингвинов (столбец `species`).

```
# Сначала получаем распределение через агрегацию (лениво)
species_dist = df['species'].value_counts()

# Выполняем вычисление
with ProgressBar():
    species_counts = species_dist.compute()

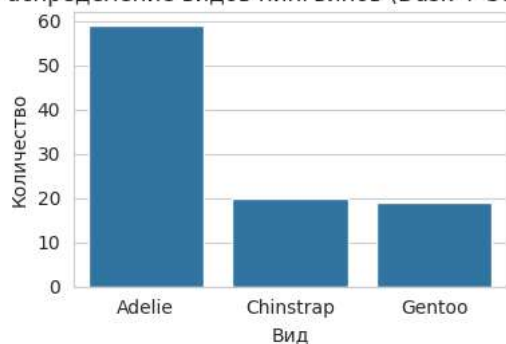
# Преобразуем в Pandas Series для удобства
species_counts = species_counts.sort_index()

# Визуализация
plt.figure(figsize=(4, 3))
sns.barplot(x=species_counts.index, y=species_counts.values)
plt.title('Распределение видов пингвинов (Dask + Seaborn)')
plt.ylabel('Количество')
plt.xlabel('Вид')
plt.tight_layout()
plt.show()

# Вывод таблицы
print("Количество пингвинов по видам:")
print(species_counts)
```

[#####] | 100% Completed | 238.47 ms

Распределение видов пингвинов (Dask + Seaborn)



Количество пингвинов по видам:
species
Adelie 59
Chinstrap 20
Gentoo 19
Name: count, dtype: int64[pyarrow]

✓ 4. Работа с пропусками и значениями

✓ Пропуски по строкам

Аналог `data[data.isna().any(axis=1)]` в Pandas. В Dask нет прямого аналога `axis=1` для фильтрации строк с любыми пропусками, но можно использовать комбинацию методов.

```
# Найдем строки, где хотя бы в одном столбце есть пропуск
# Создаем булеву маску: True, если в строке есть хотя бы один пропуск
mask = df.isnull().any(axis=1)

# Применяем маску для фильтрации строк
rows_with_nulls = df[mask]

# Выполняем вычисление
with ProgressBar():
    rows_with_nulls_computed = rows_with_nulls.compute()

print(f"Найдено строк с пропусками: {len(rows_with_nulls_computed)}")
print("Первые 5 строк с пропусками:")
print(rows_with_nulls_computed.head())
```

[#####] | 100% Completed | 112.09 ms

Найдено строк с пропусками: 28

Первые 5 строк с пропусками:

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	\
0	<NA>	Torgersen	49.819973	16.987646	204.0	
1	Adelie	Biscoe	38.599881	17.102706	204.0	
6	Adelie	Biscoe	46.110708	15.727950	NaN	
8	Chinstrap	Dream	49.647183	18.306842	172.0	
11	Adelie	Biscoe	NaN	15.295320	200.0	

body_mass_g sex

```

0      3565.0 <NA>
1      4114.0 <NA>
6      3959.0 MALE
8      3613.0 <NA>
11     3587.0 <NA>

```

Удаление строк с более чем 1 пропуском

Аналог `data = data[data.isna().sum(axis=1) <= 1]` в Pandas.

```

# Подсчитываем количество пропусков в каждой строке
null_counts_per_row = df.isnull().sum(axis=1)

# Фильтруем: оставляем строки, где количество пропусков <= 1
df_clean = df[null_counts_per_row <= 1]

# Проверим, сколько строк осталось
with ProgressBar():
    num_rows_clean = len(df_clean)
    num_rows_original = len(df)

print(f"Исходное количество строк: {num_rows_original}")
print(f"Количество строк после очистки: {num_rows_clean}")
print(f"Удалено строк: {num_rows_original - num_rows_clean}")

[#####] | 100% Completed | 108.71 ms
[#####] | 100% Completed | 104.05 ms
Исходное количество строк: 100
Количество строк после очистки: 95
Удалено строк: 5

```

```

# Проверим, остались ли строки с пропусками
mask_after_clean = df_clean.isnull().any(axis=1)
with ProgressBar():
    remaining_null_rows = df_clean[mask_after_clean].compute()

print(f"Осталось строк с пропусками: {len(remaining_null_rows)}")
if len(remaining_null_rows) > 0:
    print("Примеры таких строк:")
    print(remaining_null_rows.head())

[#####] | 100% Completed | 113.24 ms
Осталось строк с пропусками: 23
Примеры таких строк:
   species  island  bill_length_mm  bill_depth_mm  flipper_length_mm \
1   Adelie  Biscoe      38.599881      17.102706             204.0
6   Adelie  Biscoe      46.110708      15.727950              NaN
8  Chinstrap  Dream      49.647183      18.306842             172.0
12  Gentoo   Dream      41.462780      19.578934             209.0
13 Chinstrap  Dream      45.657497           NaN             182.0

   body_mass_g  sex
1      4114.0 <NA>
6      3959.0 MALE
8      3613.0 <NA>
12     3692.0 <NA>
13     4865.0 MALE

```

Удаление строк, где все значения пропущены

Аналог `df.dropna(how='all')` в Pandas.

```

# Метод .dropna() в Dask ведет себя аналогично Pandas
df_no_all_nulls = df.dropna(how='all')

# Сравним размеры
with ProgressBar():
    original_size = len(df)
    cleaned_size = len(df_no_all_nulls)

print(f"Исходное количество строк: {original_size}")
print(f"После удаления полностью пустых строк: {cleaned_size}")

[#####] | 100% Completed | 107.86 ms
[#####] | 100% Completed | 111.44 ms
Исходное количество строк: 100
После удаления полностью пустых строк: 100

```

5. Разведочный анализ данных (EDA)

Корреляция категориальных переменных с целевой

Построим круговые диаграммы для визуализации распределения видов пингинов (`species`) в зависимости от острова (`island`) и пола (`sex`).

```
# Определим категориальные столбцы
cat_cols = ['island', 'sex']
target_col = 'species'

# Используем очищенные данные (df_clean)
df_for_analysis = df_clean

# Для каждой категориальной переменной построим визуализацию
for col_name in cat_cols:
    # Группируем по категориальной переменной и целевой, подсчитываем количество
    # Dask поддерживает операции группировки, аналогичные Pandas[citation:3]
    grouped = df_for_analysis.groupby([col_name, target_col]).size()

    # Выполняем вычисление
    with ProgressBar():
        grouped_computed = grouped.compute()

    # Преобразуем в удобный формат для визуализации
    # Reset index для преобразования в DataFrame
    grouped_df = grouped_computed.reset_index(name='count')

    # Получаем уникальные значения категории
    unique_vals = grouped_df[col_name].dropna().unique()

    # Создаем subplot
    fig, axes = plt.subplots(1, len(unique_vals), figsize=(5*len(unique_vals), 4))
    # Если только одно значение, axes не будет массивом
    if len(unique_vals) == 1:
        axes = [axes]

    fig.suptitle(f'Распределение {target_col} по значениям {col_name}', fontsize=14)

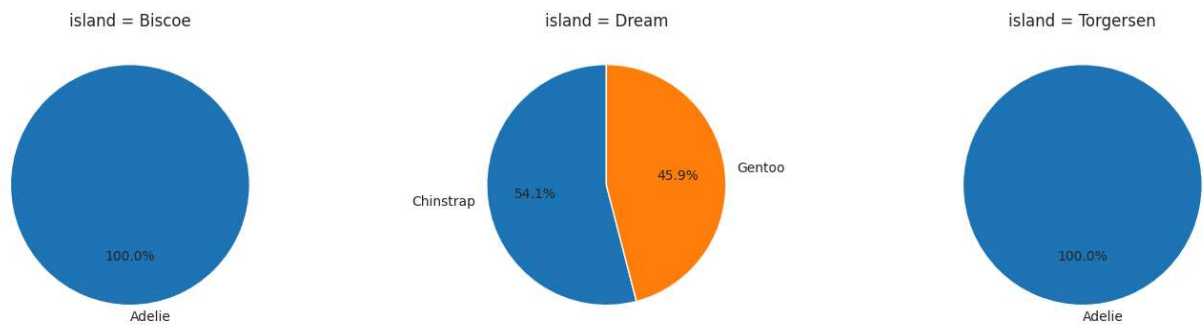
    for idx, val in enumerate(unique_vals):
        # Фильтруем данные для текущего значения
        temp = grouped_df[grouped_df[col_name] == val]

        # Строим круговую диаграмму
        axes[idx].pie(temp['count'], labels=temp[target_col], autopct='%1.1f%%', startangle=90)
        axes[idx].set_title(f'{col_name} = {val}')

plt.tight_layout()
plt.show()
```

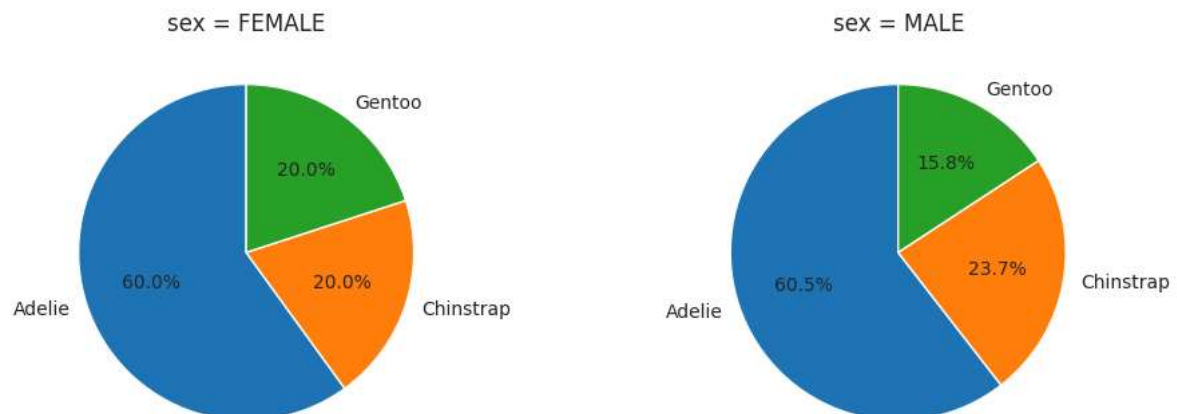
[#####] | 100% Completed | 232.41 ms

Распределение species по значениям island



[#####] | 100% Completed | 103.01 ms

Распределение species по значениям sex



✓ Анализ числовых переменных

Для числовых столбцов построим гистограммы и ящики с усами (box plot).

```
# Определим числовые столбцы
num_cols = ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', 'body_mass_g']

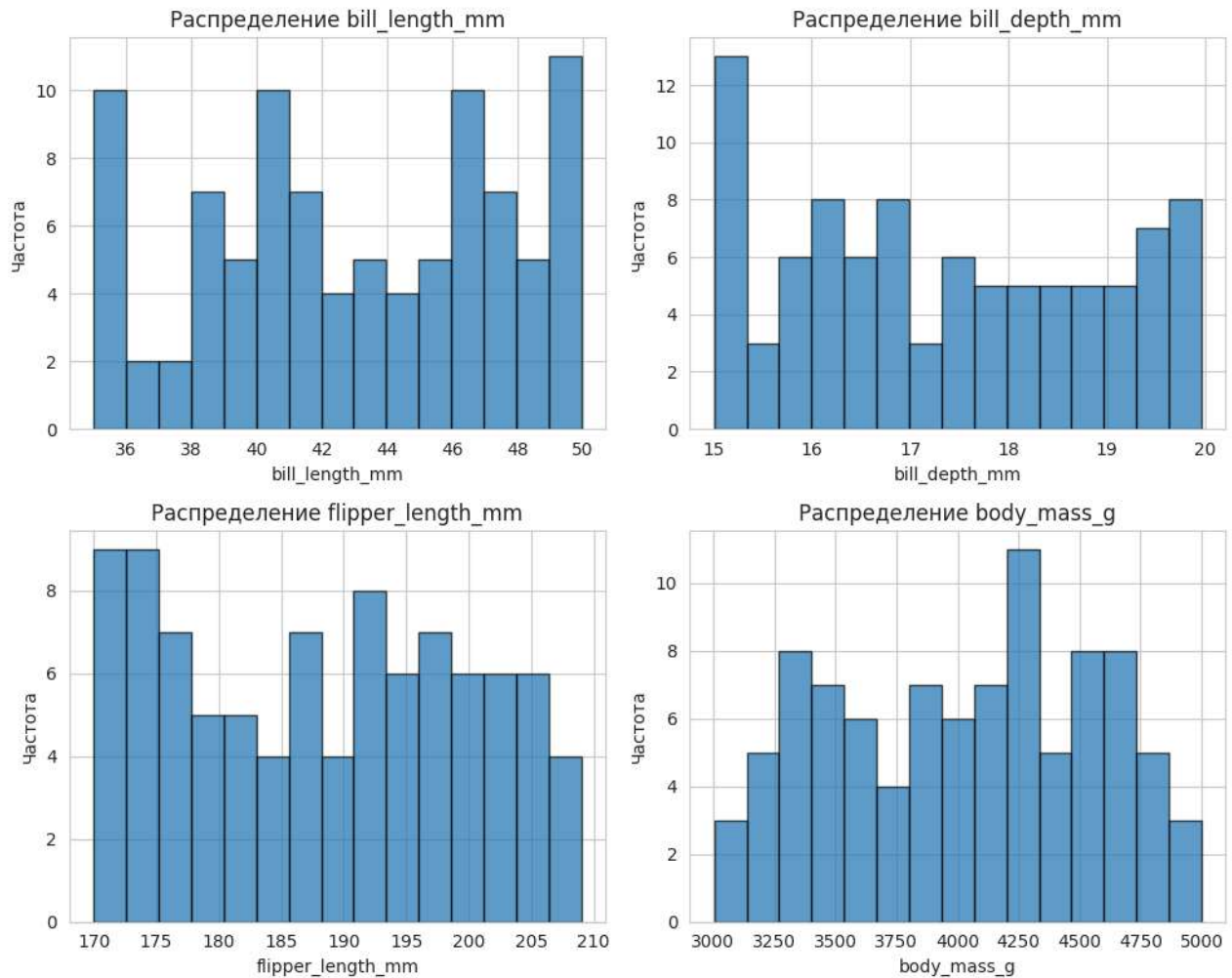
# Для визуализации нам нужно вычислить данные
with ProgressBar():
    numeric_data = df_for_analysis[num_cols].compute()

# Построим гистограммы для каждого числового признака
fig, axes = plt.subplots(2, 2, figsize=(10, 8))
axes = axes.flatten()

for idx, col in enumerate(num_cols):
    axes[idx].hist(numeric_data[col].dropna(), bins=15, alpha=0.7, edgecolor='black')
    axes[idx].set_title(f'Распределение {col}')
    axes[idx].set_xlabel(col)
    axes[idx].set_ylabel('Частота')

plt.tight_layout()
plt.show()
```

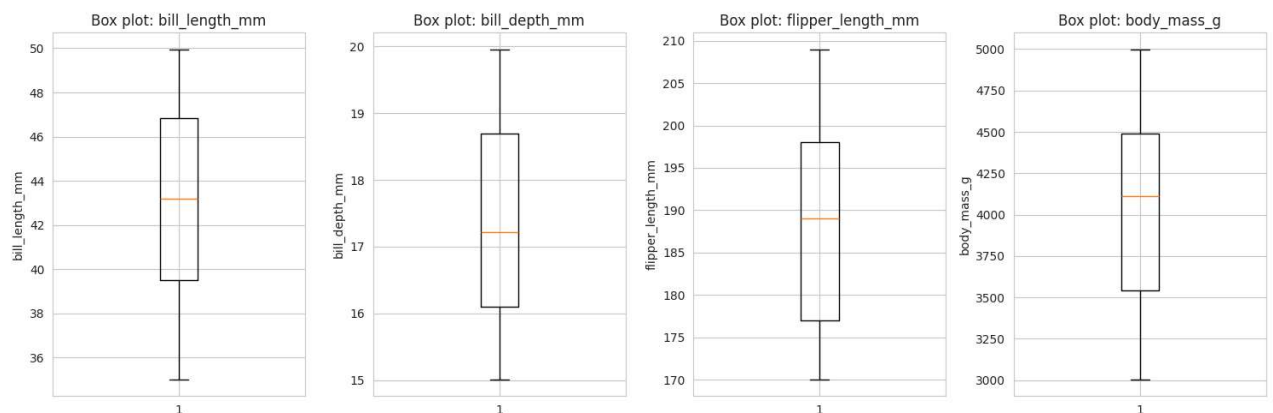

[#####] | 100% Completed | 102.70 ms



```
# Построим ящики с усами (box plot)
fig, axes = plt.subplots(1, len(num_cols), figsize=(15, 5))

for idx, col in enumerate(num_cols):
    axes[idx].boxplot(numeric_data[col].dropna())
    axes[idx].set_title(f'Box plot: {col}')
    axes[idx].set_ylabel(col)

plt.tight_layout()
plt.show()
```



Статистическое описание числовых переменных

Аналог `df.describe()` в Pandas.

```
# Dask также имеет метод .describe()
with ProgressBar():
```

```
description = df_for_analysis[num_cols].describe().compute()

print("Статистическое описание числовых переменных:")
print(description)
```

[#####] | 100% Completed | 208.04 ms

Статистическое описание числовых переменных:

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
count	94.000000	93.000000	93.000000	93.000000
mean	42.986762	17.362811	188.129032	4020.569892
std	4.562790	1.559927	11.579804	532.757641
min	35.010298	15.008792	170.000000	3004.000000
25%	40.481619	16.101306	182.500000	3852.000000
50%	43.774525	17.852728	188.000000	4276.000000
75%	47.442865	19.326762	198.750000	4545.500000
max	49.954532	19.959031	209.000000	4999.000000

6. Заключение и сравнение Dask с PySpark

Ключевые отличия Dask от PySpark, которые мы увидели:

- Синтаксис и интеграция:** Dask максимально повторяет API Pandas, что упрощает переход для тех, кто уже знаком с Pandas. PySpark имеет собственный API, отличный от Pandas.
- Ленивые вычисления:** Обе библиотеки используют ленивые вычисления, но в Dask для получения результатов почти всегда требуется явный вызов `.compute()` или `.persist()`.
- Архитектура:** PySpark изначально заточен под распределенные кластеры, Dask может работать как на одном компьютере (параллельно), так и на кластере.
- Производительность:** Для очень больших данных, распределенных по кластерам, PySpark часто показывает лучшую производительность благодаря оптимизированному движку Tungsten и Catalyst. Dask отлично подходит для данных, которые не помещаются в память одного компьютера, но обрабатываются на нем же.
- Экосистема:** Dask тесно интегрирован с экосистемой Python (NumPy, Pandas, Scikit-learn), позволяя параллелить существующие workflows. PySpark имеет собственную экосистему (MLlib, GraphX) и сильнее интегрирован с Hadoop.

Когда выбирать Dask:

- Работа в экосистеме Python, использование Pandas/NumPy/Scikit-learn.
- Данные не помещаются в оперативную память, но кластер недоступен или избыточен.

